

PERTEMUAN KE-4: PREPROCESSING DATA

A. TUJUAN

Setelah menempuh praktikum ini diharapkan mahasiswa mampu

1. menjelaskan tujuan dari data pre-processing.
2. melakukan pre-processing data dengan menggunakan Bahasa Pemrograman Python.

B. MATERI

Preprocessing merupakan tahapan persiapan / proses awal pengolahan data asli sebelum data dilakukan proses mining (penggalian informasi dari data yang telah diolah). *Preprocessing* ini bertujuan untuk mempersiapkan data mentah yang sebelumnya belum bisa dikategorikan belum baik. Penyebab data dikatakan belum baik dikarenakan noisy, inconsisten dan incomplete. Noisy adalah data awal berisi data yang tidak diinginkan sedangkan inconsisten merupakan data yang tidak sama pada masing-masing kategori. Incomplete adalah data yang memiliki missing value.

Data preprocessing adalah tahap penting dalam analisis data yang bertujuan untuk mempersiapkan data agar siap untuk dianalisis atau digunakan dalam model pembelajaran mesin. Ada beberapa teknik dan pustaka yang dapat digunakan untuk melakukan data preprocessing dalam Python. Berikut adalah beberapa langkah umum dalam data preprocessing dan pustaka Python yang sering digunakan:

a. Penanganan Data yang Hilang (Missing Data)

Data yang hilang dapat terjadi dalam dataset, dan perlu ditangani agar analisis atau model pembelajaran mesin tidak terganggu.

- Menghapus Nilai yang Hilang. Salah satu pendekatan adalah dengan menghapus baris atau kolom yang mengandung nilai yang hilang.
`df.dropna()` # Menghapus baris dengan nilai yang hilang
`df.dropna(axis=1)` # Menghapus kolom dengan nilai yang hilang
- Mengganti Nilai yang Hilang. Nilai yang hilang juga bisa diganti oleh nilai lain, seperti rata-rata, median, atau modus.
`df.fillna(df.mean())` # Mengisi nilai yang hilang dengan rata-rata kolom

b. Normalisasi dan Standarisasi

Normalisasi digunakan untuk membuat data lebih seragam terutama jika digunakan dalam model pembelajaran mesin, normalisasi atau standarisasi diperlukan.

- Normalisasi (Min-Max Scaling) digunakan untuk mengubah data menjadi rentang tertentu, biasanya [0, 1].
`from sklearn.preprocessing import MinMaxScaler`
`scaler = MinMaxScaler()`
`df_scaled = scaler.fit_transform(df)`
- Standarisasi (Z-Score Scaling) digunakan untuk mengubah data sehingga memiliki rata-rata 0 dan deviasi standar 1.
`from sklearn.preprocessing import StandardScaler`
`scaler = StandardScaler()`
`df_standardized = scaler.fit_transform(df)`

c. Encoding Variabel Kategorikal

Banyak algoritma pembelajaran mesin membutuhkan data numerik, jadi variabel kategorikal perlu diubah menjadi format numerik.

- Label Encoding digunakan untuk mengubah kategori menjadi angka (misalnya, "Red" menjadi 0, "Blue" menjadi 1).
`from sklearn.preprocessing import LabelEncoder`
`encoder = LabelEncoder()`
`df['Category'] = encoder.fit_transform(df['Category'])`
- One-Hot Encoding digunakan untuk mengubah kategori menjadi representasi vektor biner (misalnya, "Red" menjadi [1, 0, 0], "Blue" menjadi [0, 1, 0]).
`df = pd.get_dummies(df, columns=['Category'])`

d. Menangani Outlier (Pencilan)

Outlier atau pencilan adalah nilai yang sangat berbeda dari sebagian besar data. Mereka dapat mempengaruhi analisis dan model.

- Mengidentifikasi Outlier. Salah satu cara untuk mengidentifikasi outlier adalah menggunakan metode IQR (Interquartile Range).
 $Q1 = df.quantile(0.25)$
 $Q3 = df.quantile(0.75)$
 $IQR = Q3 - Q1$
 $outliers = ((df < (Q1 - 1.5 * IQR)) | (df > (Q3 + 1.5 * IQR)))$
- Menghapus atau Mengganti Outlier. Setelah mengidentifikasi outlier, maka outlier bisa dihapus atau diganti.

e. Transformasi Data

Kadang-kadang data perlu ditransformasikan agar lebih sesuai untuk analisis atau pemodelan.

- Log Transformation menggunakan logaritma untuk menangani data dengan distribusi yang sangat miring (skewed).
 $df['Transformed'] = np.log(df['Column'] + 1)$
- Square Root Transformation digunakan untuk data yang memiliki distribusi positif dengan varians tinggi.
 $df['Transformed'] = np.sqrt(df['Column'])$

f. Fitur Ekstraksi dan Pemilihan Fitur

Proses ini mencakup pemilihan fitur yang relevan dan membuang fitur yang tidak diperlukan.

- Pilih Fitur yang Relevan yaitu memilih kolom-kolom yang penting atau menggunakan teknik seperti feature selection. PCA (Principal Component Analysis): Teknik ini digunakan untuk mereduksi dimensi data dengan mempertahankan informasi sebanyak mungkin.
 $from sklearn.decomposition import PCA$
 $pca = PCA(n_components=2)$
 $df_pca = pca.fit_transform(df)$

g. Pembagian Data untuk Pelatihan dan Pengujian

Setelah data diproses, biasanya dibagi menjadi set pelatihan dan pengujian agar model dapat diuji dan dievaluasi dengan baik.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

h. Penggabungan dan Pengurutan Data

Penggabungan Data. Data yang berasal dari berbagai sumber bisa digabungkan data

```
df_merged = pd.merge(df1, df2, on='ID')
```

Pengurutan Data: Mengurutkan data berdasarkan kolom tertentu.

```
df_sorted = df.sort_values(by='Column', ascending=False)
```

Beberapa pustaka yang sering digunakan dalam data preprocessing di Python yaitu pandas dan Scikit-learn. Pandas menyediakan manipulasi data seperti penghapusan atau pengisian nilai yang hilang, penggabungan data. Scikit-learn menyediakan pustaka utama untuk pembelajaran mesin, yang menyediakan banyak alat untuk preprocessing, seperti normalisasi, standarisasi, dan encoding.

Contoh 4.1

Buatlah program untuk mendapatkan prosentase missing value setiap fitur pada data heart_disease_uci.csv.

Jawaban

Kode Program 4.1. Program untuk mendapatkan prosentase missing value.

No	Kode Program
1	<code>import pandas as pd</code>
2	<code>df = pd.read_csv("heart_disease_uci.csv")</code>
3	<code>missing_values = df.isnull().sum()</code>
4	<code>missing_percentage = (missing_values / len(df)) * 100</code>
5	<code># Menggabungkan hasil dalam DataFrame</code>
6	<code>missing_summary = pd.DataFrame({"Total Missing": missing_values,</code>
7	<code>"Missing Percentage (%)": missing_percentage})</code>
8	<code># Menampilkan hanya kolom dengan missing values</code>
9	<code>missing_summary = missing_summary[missing_summary["Total Missing"] > 0]</code>


```

10 # Menampilkan hasil
11 print("Persentase Missing Values per Kolom:")
12 print(missing_summary

```

Keluaran dari Kode Program 4.1 adalah

```

Persentase Missing Values per Kolom:
      Total Missing  Missing Percentage (%)
trestbps           59           6.413043
chol               30           3.260870
fbs                90           9.782609
restecg             2           0.217391
thalch             55           5.978261
exang              55           5.978261
oldpeak            62           6.739130
slope              309          33.586957
ca                 611          66.413043
thal              486          52.826087

```

Contoh 4.2

Buatlah program untuk memeriksa distribusi fitur chol normal atau tidak.

Jawaban

Kode Program 4.2. Program untuk memeriksa distribusi fitur chol normal atau tidak.

No	Kode Program
1	import pandas as pd
2	from scipy import stats
3	df = pd.read_csv("heart_disease_uci.csv")
4	df1=df
5	# Melakukan uji normalitas Shapiro-Wilk
6	stat, p_value = stats.shapiro(df1["chol"].dropna())
7	# Menampilkan hasil uji
8	print(f"Statistik uji: {stat}")
9	print(f"P-value: {p_value}")
10	# Menentukan apakah data berdistribusi normal
11	alpha = 0.05 # Level signifikansi 5%
12	if p_value > alpha:
13	print("Data berdistribusi normal (gagal menolak hipotesis nol).")
14	else:
15	print("Data tidak berdistribusi normal (menolak hipotesis nol).")

Keluaran dari Kode Program 4.2 adalah

Statistik uji: 0.865710973739624

P-value: 6.390689304571746e-27

Data tidak berdistribusi normal (menolak hipotesis nol).

Contoh 4.3

Buatlah program untuk melakukan imputasi dengan median pada fitur chol.

Jawaban

Kode Program 4.3. Program untuk melakukan imputasi dengan median pada fitur chol.

No	Kode Program
1	import pandas as pd
2	import numpy as np
3	# Menampilkan jumlah missing values sebelum imputasi
4	print("Jumlah missing values sebelum imputasi:")
5	print(df['chol'].isnull().sum())
6	# Menghitung median dari kolom 'chol'
7	median_chol = df["chol"].median()
8	# Mengisi missing values (NaN) pada kolom 'chol' dengan median
9	df["chol"] = df["chol"].fillna(median_chol)
10	# Menampilkan jumlah missing values setelah imputasi
11	print("\nJumlah missing values setelah imputasi:")
12	print(df['chol'].isnull().sum())

Keluaran dari Kode Program 4.3 adalah

Jumlah missing values sebelum imputasi:

30

Jumlah missing values setelah imputasi:

0

Contoh 4.4

Buatlah program untuk melakukan hot encoding pada fitur sex.

Jawaban

Kode Program 4.4. Program untuk melakukan hot encoding pada fitur sex.

No	Kode Program
1	import pandas as pd
2	from sklearn.preprocessing import OneHotEncoder
3	df = pd.read_csv("heart_disease_uci.csv")
4	# Inisialisasi OneHotEncoder
5	encoder = OneHotEncoder(sparse_output=False) # sparse=False untuk
6	mendapatkan hasil dalam bentuk array dense
7	# Melakukan one-hot encoding pada kolom 'sex'
8	encoded_sex = encoder.fit_transform(df[["sex"]])
9	# Membuat DataFrame baru dari hasil encoding
10	encoded_df = pd.DataFrame(encoded_sex,
11	columns=encoder.get_feature_names_out(["sex"]))
12	# Menambahkan hasil encoding ke DataFrame awal
13	df_encoded = pd.concat([df, encoded_df], axis=1)
14	# Menampilkan data setelah one-hot encoding
15	print("\nData setelah one-hot encoding:")
16	df_encoded.head()

Contoh 4.5

Buatlah program untuk melakukan min max normalization.

Jawaban

Kode Program 4.5. Program untuk melakukan min max normalization.

No	Kode Program
1	import pandas as pd
2	from sklearn.preprocessing import MinMaxScaler
3	data = {
4	"age": [23, 45, 56, 67, 34],
5	"chol": [220, 240, 210, 180, 250],
6	"trestbps": [130, 150, 120, 140, 160]
7	}
8	df = pd.DataFrame(data)
9	# Menampilkan data sebelum normalisasi
10	print("Data sebelum normalisasi:")
11	print(df)
12	# Inisialisasi MinMaxScaler
13	scaler = MinMaxScaler()
14	# Melakukan min-max normalization pada seluruh data
15	df_normalized = scaler.fit_transform(df)
16	# Membuat DataFrame baru dengan data yang sudah dinormalisasi
17	df_normalized = pd.DataFrame(df_normalized, columns=df.columns)
18	# Menampilkan data setelah normalisasi
19	print("\nData setelah normalisasi (min-max):")
20	print(df_normalized)

Contoh 4.6

Buatlah program untuk menghitung korelasi antar fitur dan menggambar diagram heatmap.

Jawaban

Kode Program 4.6. Program untuk menghitung korelasi antar fitur dan menggambar diagram heatmap.

No	Kode Program
1	import pandas as pd
2	import seaborn as sns
3	import matplotlib.pyplot as plt
4	data = {
5	"age": [23, 45, 56, 67, 34],
6	"chol": [220, 240, 210, 180, 250],
7	"trestbps": [130, 150, 120, 140, 160],
8	"oldpeak": [1.5, 2.3, 1.8, 2.2, 1.9]


```

9     }
10    df = pd.DataFrame(data)
11    # Menghitung korelasi antar fitur
12    correlation_matrix = df.corr()
13    # Menampilkan korelasi antar fitur
14    print("Korelasi antar fitur:")
15    print(correlation_matrix)
16    # Membuat heatmap untuk visualisasi korelasi
17    plt.figure(figsize=(8, 6))
18    sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f',
19               linewidths=0.5)
20    plt.title("Heatmap Korelasi Antar Fitur")
21    plt.show()

```

C. TUGAS/LATIHAN SOAL

1. Pada dataset `heart_disease_uci.csv`, beberapa kolom memiliki nilai yang hilang (missing values). Lakukan langkah-langkah berikut:
 - a. Periksa kolom mana saja yang memiliki missing values.
 - b. Lakukan imputasi untuk kolom "trestbps" dan "chol" menggunakan median.
 - c. Pilih kolom "slope" dan "thal", yang bersifat kategorikal, lalu lakukan imputasi menggunakan mode (nilai kategori yang paling sering muncul).
 - d. Setelah imputasi, periksa kembali dataset untuk memastikan bahwa tidak ada missing values yang tersisa.
2. Pada dataset `heart_disease_uci.csv`, terdapat beberapa kolom yang bersifat kategorikal seperti "sex", "dataset", "cp", "restecg", dan "thal". Lakukan langkah-langkah berikut:
 - a. Lakukan one-hot encoding pada kolom "sex" (misalnya, buat dua kolom: "sex_Male" dan "sex_Female").
 - b. Lakukan Label Encoding pada kolom "dataset" untuk mengubah kategori "Cleveland", "Hungarian", dll. menjadi angka.
3. Hitung korelasi antar fitur numerik seperti "age", "trestbps", "chol", "thalch", "oldpeak", dll. Visualisasikan korelasi antar fitur menggunakan heatmap dengan pustaka Seaborn. Tentukan fitur mana yang paling berkorelasi dengan kolom "num" (kolom target yang menunjukkan apakah seseorang memiliki penyakit jantung atau tidak).
4. Terdapat fitur numerik dalam dataset seperti "age", "trestbps", "chol", dan "thalch". Lakukan langkah-langkah berikut:
 - a. Min-Max Normalization pada kolom "age" dan "trestbps" menggunakan `MinMaxScaler`.
 - b. Lakukan standarisasi (z-score normalization) pada kolom "chol" dan "thalch" menggunakan `StandardScaler`.
 - c. Jelaskan perbedaan antara normalisasi dan standarisasi serta kapan Anda akan memilih salah satu metode tersebut.
5. Pada dataset `heart_disease_uci.csv`, kolom "ca" dan "slope" memiliki banyak missing values. Lakukan langkah-langkah berikut:
 - a. Cek persentase missing values pada kolom "ca" dan "slope".
 - b. Keputusan: Jika kolom tersebut memiliki lebih dari 50% nilai yang hilang, sebaiknya kolom tersebut dihapus atau tetap dipertahankan dengan imputasi.
 - c. Lakukan imputasi atau hapus kolom tersebut berdasarkan keputusan Anda.
6. Pada kolom "chol" dan "trestbps", periksa apakah terdapat outliers:
 - a. Gunakan boxplot untuk memeriksa kemungkinan adanya outliers pada kedua kolom tersebut.
 - b. Tentukan apakah akan menghapus outliers atau mentransformasi data tersebut untuk menangani masalah ini.
7. Dataset perlu dipisahkan menjadi fitur (X) dan target (y) untuk membangun model
 - a. Pisahkan kolom "num" (target) dan kolom lainnya (fitur).
 - b. Pisahkan dataset menjadi data latih (train) dan data uji (test) menggunakan `train_test_split` dari pustaka `sklearn` dengan proporsi 80% untuk data latih dan 20% untuk data uji.